Анализ данных с использованием языка Python

Пояснения к коду. Семинар 3

NumPy: случайные выборки, статистики, пропуски, поиск и сортировка

А. Н. Лата

Пояснения к ноутбуку `notebooks/seminar-03-numpy-stats.ipynb`. Номер ячейки — её порядковый номер среди ячеек кода ноутбука, считая сверху. Для каждой ячейки приведён её код с номерами строк, а под ним — что делает каждая команда и что означает каждый аргумент именно в этой ячейке. Команды, разобранные в пояснениях к семинарам 1–2, здесь только называются. Код совпадает с ноутбуком, только пробелы перед комментариями сокращены. Полные подписи функций со всеми параметрами — в справочнике `reference/numpy.md`.

1 Случайные числа и выборки

Ячейка 1. Генератор с зерном

```
1 import numpy as np
2
3 rng = np.random.default_rng(3)
4 rng.integers(10_000, 1_000_000, size=5)  # пять случайных цен исков
```

– `np.random.default_rng(3)` — генератор с зерном 3. Генератор создаётся в этой же ячейке, поэтому повторный запуск ячейки даёт те же числа.

– `rng.integers(10_000, 1_000_000, size=5)` — пять случайных целых от 10 000 до 999 999, как на семинаре 2.

Ячейка 2. Выборка без повторов

```
1 numbers = np.arange(1, 13)  # номера дел
2
3 rng.choice(numbers, size=3, replace=False)  # три дела на проверку
```

– `np.arange(1, 13)` — номера дел от 1 до 12.

– `rng.choice(numbers, size=3, replace=False)` — три случайных элемента массива `numbers`. `replace=False` — без возвращения: каждый элемент может попасть в выборку только один раз.

Ячейка 3. Выборка с возвращением

```
1 rng.choice(numbers, size=4)  # с возвращением: повторы возможны
```

– `rng.choice(numbers, size=4)` — без `replace=False` выбор с возвращением, это значение по умолчанию: одно и то же дело может попасть несколько раз.

Ячейка 4. Случайный порядок

```
order = rng.permutation(numbers)  # дела в случайном порядке
order
```

– `rng.permutation(numbers)` — новый массив из тех же элементов в случайном порядке; каждый элемент ровно один раз. Сам `numbers` не меняется.

Ячейка 5. Три судьи

```
order[:4], order[4:8], order[8:]  # дела трёх судей
```

– `order[:4]`, `order[4:8]`, `order[8:]` — три среза по четыре элемента: первые четыре, с пятого по восьмой и с девятого до конца. Правая граница каждого среза — начало следующего.

Ячейка 6. Тот же генератор заново

```
np.random.default_rng(3).integers(10_000, 1_000_000, size=5)  # те же пять ↩
↪ цен
```

– `np.random.default_rng(3).integers(...)` — новый генератор с тем же зерном, без переменной, и сразу вызов `integers`: цепочка начинается заново, числа те же, что в ячейке 1.

Ячейка 7. Генератор без зерна

```
np.random.default_rng().integers(10_000, 1_000_000, size=5)  # каждый раз ↩
↪ новые
```

– `np.random.default_rng()` без зерна — при каждом запуске новые числа.

2 Картотека на 1000 дел

Ячейка 8. Картотека

```
rng = np.random.default_rng(2026)

amounts = np.append(rng.integers(10_000, 300_000, size=950),
                    rng.integers(1_000_000, 5_000_000, size=50))  # цены ↩
↪ исков, руб.
rng.shuffle(amounts)  # перемешать на месте
days = rng.integers(10, 300, size=1000)  # срок рассмотрения, дней
outcomes = rng.integers(0, 2, size=1000)  # 1 удовлетворён, 0 отказано
numbers = np.arange(1, 1001)  # номера дел

amounts
```

– Строки 3–4 — `np.append(a, b)`: массив из двух частей, 950 обычных цен до 300 000 и 50 крупных от одного до пяти миллионов. Вызов записан на двух строках: пока скобка не закрыта, Python ждёт продолжения.

– `rng.shuffle(amounts)` — перемешать сам массив, «на месте»: функция ничего не возвращает, а меняет `amounts`. Без этого все крупные иски стояли бы в конце.

– Строки 6–8 — сроки от 10 до 299 дней, исходы 0 или 1 и номера дел от 1 до 1000.

– `amounts` последней строкой — показать массив. Тысячу элементов NumPy сокращает: начало, многоточие и конец.

Ячейка 9. Размер и крайние значения

```
amounts.size, amounts.min(), amounts.max()
```

– `size`, `min()`, `max()` — число элементов, наименьшая и наибольшая цена.

3 Описательные статистики

Ячейка 10. Среднее и медиана

```
amounts.mean(), np.median(amounts)  # среднее и медиана
```

– `amounts.mean()` — среднее, метод массива. `np.median(amounts)` — медиана, функция NumPy: массив передаётся в скобках.

Ячейка 11. Медиана методом: намеренная ошибка

```
amounts.median()
```

– У массива нет метода `median`, поэтому `AttributeError`: «у объекта нет атрибута». Медиана — только `np.median(...)`.

Ячейка 12. Стандартное отклонение

```
amounts.std()  # разброс цен
```

– `amounts.std()` — стандартное отклонение: типичное расстояние значений от среднего, в тех же единицах, что цены.

Ячейка 13. Процентиль

```
np.percentile(days, 90)  # срок 90 % дел
```

– `np.percentile(days, 90)` — 90-й процентиль: значение, не больше которого 90 процентов элементов.

Ячейка 14. Квартили

```
np.percentile(amounts, [25, 50, 75])  # квартили цен
```

– `np.percentile(amounts, [25, 50, 75])` — список процентилей вместо одного числа даёт массив из трёх значений: первый квартиль, медиану и третий квартиль.

Ячейка 15. Доля и средний срок

```
outcomes.mean(), days.mean()  # доля удовлетворённых, средний срок
```

– `outcomes.mean()` — доля единиц, как на семинаре 2; `days.mean()` — средний срок.

4 Пропуски

Ячейка 16. Незавершённые дела

```
unfinished = rng.choice(1000, size=50, replace=False)  # индексы ↩
↪ незавершённых дел

days = days.astype(float)
outcomes = outcomes.astype(float)
days[unfinished] = np.nan
outcomes[unfinished] = np.nan

print(days[:10])  # начало картотеки
days[unfinished[:3]]  # сроки трёх незавершённых дел
```

– `rng.choice(1000, size=50, replace=False)` — число 1000 вместо массива: выбор из позиций от 0 до 999. Это 50 случайных позиций без повторов.

– `days.astype(float)` — сроки в дробный тип: `nan` бывает только у дробных чисел. Результат записан в ту же переменную.

– `days[unfinished] = np.nan` — массив позиций в квадратных скобках: записать пропуск во все 50 позиций сразу. `np.nan` — пропуск, «не число».

– Строки 8–9 — начало картотеки и сроки трёх первых незавершённых дел: `unfinished[:3]` — три первые позиции из списка.

Ячейка 17. Пропуск не равен себе

```
np.nan == np.nan  # nan не равен даже себе
```

– `np.nan == np.nan` — сравнение с пропуском всегда даёт `False`. Поэтому `days == np.nan` не находит ни одного пропуска.

Ячейка 18. Поиск пропусков

```
np.isnan(days).sum(), np.isnan(days).mean()  # число и доля незавершённых
```

– `np.isnan(days)` — маска: `True` там, где пропуск. Её сумма — число пропусков, среднее — их доля.

Ячейка 19. Среднее с пропусками

```
days.mean()  # nan: пропуски ломают среднее
```

– `days.mean()` — любая арифметика с `nan` даёт `nan`, и среднее тоже. Ошибки нет.

Ячейка 20. Статистики без пропусков

```
np.nanmean(days), np.nanmedian(days), np.nanpercentile(days, 90)  # без ↩
↪ пропусков
```

– `np.nanmean`, `np.nanmedian`, `np.nanpercentile` — те же статистики, но по известным значениям: пропуски пропускаются.

Ячейка 21. То же через маску

```
days[~np.isnan(days)].mean()  # то же через маску
```

– `~np.isnan(days)` — перевёрнутая маска: `True` там, где срок известен. `days[...]` — только известные сроки, и среднее считается обычным `mean()`.

Ячейка 22. Заполнение медианой

```
filled = days.copy()
filled[np.isnan(filled)] = np.nanmedian(days)

np.isnan(filled).sum(), filled.mean()  # пропусков нет, среднее считается
```

– `filled = days.copy()` — копия: заполнять будем её, картотека останется с пропусками.

– `filled[np.isnan(filled)] = np.nanmedian(days)` — запись по маске, как на семинаре 2: во все пропуски одно число, медиану известных сроков.

– Строка 4 — проверка: пропусков в копии 0, и обычное среднее считается.

5 Поиск

Ячейка 23. Значение по условию

```
procedure = np.where(amounts <= 100_000, 'упрощённое', 'общее')  # порядок ↩
↪ производства
procedure[10:20]
```

– `np.where(условие, a, b)` — новый массив: где условие верно — `a`, где нет — `b`. Здесь `a` и `b` — строки, и число превращается в категорию.

– `procedure[10:20]` — показать элементы с позиции 10 по 19, то есть дела с одиннадцатого по двадцатое.

Ячейка 24. Сколько упрощённых

```
(procedure == 'упрощённое').sum()  # дел в упрощённом порядке
```

– `(procedure == 'упрощённое').sum()` — маска по строке и её сумма: число дел в упрощённом порядке.

Ячейка 25. Позиция наибольшего

```
np.argmax(amounts), amounts.max()  # индекс и значение
```

– `np.argmax(amounts)` — позиция наибольшего элемента, а не он сам; `amounts.max()` — сам наибольший элемент.

Ячейка 26. Номер самого дорогого дела

```
numbers[np.argmax(amounts)]  # номер самого дорогого дела
```

– `numbers[np.argmax(amounts)]` — позиция из одного массива подставляется в другой: номер дела на той же позиции.

Ячейка 27. argmax на пропусках

```
days[np.argmax(days)]  # argmax указывает на nan
```

– `np.argmax(days)` — при пропусках указывает на позицию первого `nan`, и `days[...]` возвращает пропуск.

Ячейка 28. Наибольшее без пропусков

```
numbers[np.nanargmax(days)], np.nanmax(days)  # самое долгое завершённое дело
```

– `np.nanargmax(days)` — позиция наибольшего среди известных значений.

– `np.nanmax(days)` — само это значение, наибольший известный срок.

6 Сортировка

Ячейка 29. Сортировка по возрастанию

```
np.sort(amounts)[:5]  # пять самых дешёвых цен
```

– `np.sort(amounts)` — отсортированная копия по возрастанию; `[:5]` — первые пять.

Ячейка 30. По убыванию

```
np.sort(amounts)[::-1][:5]  # пять самых дорогих цен
```

– `[::-1]` — срез с шагом минус один: тот же массив задом наперёд. После разворота `[:5]` — пять самых дорогих цен.

Ячейка 31. Позиции в порядке цены

```
order = np.argsort(amounts)[::-1]  # индексы по убыванию цены
numbers[order[:5]]  # номера пяти самых дорогих дел
```

– `np.argsort(amounts)` — позиции элементов в порядке возрастания цены: на первом месте — позиция самой дешёвой. `[::-1]` — разворот, от самой дорогой.

– `numbers[order[:5]]` — массив позиций в квадратных скобках: номера дел на пяти первых позициях порядка.

Ячейка 32. Сроки по тому же порядку

```
days[order[:5]]  # сроки этих пяти дел
```

– `days[order[:5]]` — те же позиции выбирают сроки тех же пяти дел.

Ячейка 33. Пропуски в сортировке

```
np.sort(days)[-3:]  # nan в конце
```

– `np.sort(days)[-3:]` — три последних элемента сортировки. Пропуски `np.sort` ставит в конец, поэтому это `nan`, а не самые долгие сроки.

7 Мини-анализ картотеки

Ячейка 34. Паспорт картотеки

```
print('Дел в картотеке:', numbers.size)
print('Не завершено:', int(np.isnan(days).sum()))
print('Типичная цена иска, руб.:', np.median(amounts))
print('Средняя цена иска, руб.:', round(amounts.mean()))
print('Четверть дел дешевле, руб.:', np.percentile(amounts, 25))
print('Типичный срок, дней:', np.nanmedian(days))
print('Разброс сроков, дней:', round(np.nanstd(days)))
print('90 % дел быстрее, дней:', np.nanpercentile(days, 90))
print('Удовлетворено исков, доля:', round(np.nanmean(outcomes), 2))
```

– `print('текст', значение)` — несколько значений через запятую печатаются в одну строку через пробел.

– `int(np.isnan(days).sum())` — число пропусков как обычное целое: `int` убирает пометку типа NumPy.

– `round(np.nanstd(days))` — стандартное отклонение сроков без пропусков, до целого; `round(..., 2)` — до двух знаков после точки.

Ячейка 35. Цена незавершённых и завершённых

```
unfinished_mask = np.isnan(days)
np.median(amounts[unfinished_mask]), np.median(amounts[~unfinished_mask])
↪ # цена незавершённых и завершённых
```

– `unfinished_mask = np.isnan(days)` — маска незавершённых дел; маска по срокам отбирает цены.

– `amounts[~unfinished_mask]` — цены завершённых дел: перевёрнутая маска.

Ячейка 36. Незавершённые среди дорогих

```
top50 = np.argsort(amounts)[::-1][:50]  # индексы 50 самых дорогих дел
np.isnan(days[top50]).mean()  # доля незавершённых среди них
```

– `np.argsort(amounts)[::-1][:50]` — позиции 50 самых дорогих дел: порядок по убыванию и первые пятьдесят.

– `np.isnan(days[top50]).mean()` — сроки этих дел, маска пропусков и её среднее: доля незавершённых.

Ячейка 37. Самые долгие дела

```
long = days > np.nanpercentile(days, 95)  # дела дольше 95 % остальных
numbers[long].size, numbers[long][:10]  # сколько их и первые десять
```

– `days > np.nanpercentile(days, 95)` — маска дел дольше 95-го процентиля. Сравнение пропуска с числом даёт `False`, поэтому незавершённые в маску не попадают.

– `numbers[long].size` — сколько таких дел; `numbers[long][:10]` — номера первых десяти: отбор маской, а затем срез.